

ECE4760 Lab 2: Digital Galton Board

Stephen Barlett, Jamayne Gyimah-Danquah, Md Shad

October 23, 2025

1 Introduction

In this lab, we implemented a digital Galton Board on the RP2040 microcontroller to model the statistical behavior of balls undergoing repeated binary interactions. Each ball fell through a triangular array of pegs, with every collision representing a Bernoulli trial that caused the ball to deflect left or right with equal probability. As more collisions occurred, the resulting distribution of balls across the bins at the bottom of the board approximated the Gaussian curve predicted by the Central Limit Theorem. Each collision triggered a “thunk” sound generated by a Direct Memory Access (DMA) channel streaming samples to an SPI-connected MCP4822 DAC, minimizing CPU overhead for audio output.

The VGA display rendered the Galton Board and a real-time histogram showing the distribution of outcomes. It also displayed key parameters such as the number of active balls, total balls dropped, bounciness coefficient, and elapsed time. A potentiometer and pushbutton provided user control, with the potentiometer adjusting either the number of balls or the coefficient of restitution depending on the current mode. Each adjustment reset the histogram to allow for new trials under different conditions.

Although we ran the program entirely on a single RP2040 core, careful optimization of arithmetic routines, data structures, and DMA operations ensured that the system was neither performance nor memory constrained. The final implementation successfully visualized the emergence of a Gaussian distribution from simple random processes while demonstrating efficient use of numerical computation, real-time rendering, and peripheral integration on a resource-limited embedded platform.

2 Design and Testing

2.1 Overview

The hardware design and testing of the digital Galton Board involved integrating multiple subsystems — VGA display output, audio generation via DAC, potentiometer-based control input, and a button interface for user mode toggling. The entire system was built around the Raspberry Pi Pico microcontroller. The design process followed a structured, incremental development approach — beginning with hardware assembly and basic signal verification, followed by integration and final validation.

Each subsystem was tested both in isolation and as part of the integrated system to ensure performance consistency, reliability, and full compliance with the lab requirements. The end goal was to achieve a visually and aurally responsive simulation at 30 frames per second (fps), where balls visually bounce off pegs while producing synchronized “thunk” audio tones. The user could dynamically adjust the number of balls and bounciness using a potentiometer, toggling between parameters with a debounced button press.

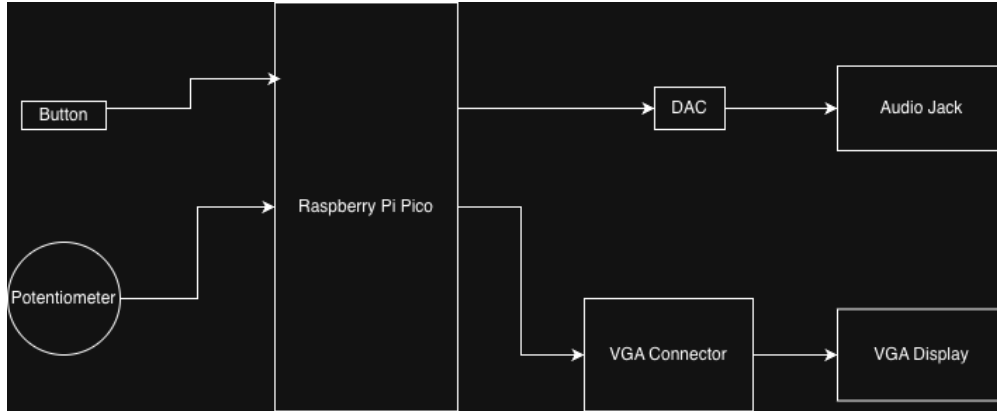


Figure 1: High-level hardware setup showing the Raspberry Pi Pico connected to the VGA circuit, DAC audio output, potentiometer, and control button.

2.2 Hardware Setup

2.2.1 VGA Output Interface

To generate the VGA display, a VGA extender PCB was soldered to connect the Raspberry Pi Pico to a VGA D-sub connector through a resistive DAC network. The board contained through-hole components including resistors, header pins, and the D-sub connector itself. The following GPIO pins were used:

- GPIO 18 → Green (Lo bit) through a 470 Ω resistor
- GPIO 19 → Green (Hi bit) through a 330 Ω resistor
- GPIO 20 → Blue through a 330 Ω resistor
- GPIO 21 → Red through a 330 Ω resistor
- GPIO 16 → HSYNC
- GPIO 17 → VSYNC

Each resistor was soldered onto the PCB, and continuity testing was performed afterward to ensure that all connections from the Pico GPIOs to the VGA D-sub pins were electrically sound. Optical inspection under bright light was carried out to identify any potential cold solder joints, which could cause signal instability. Any dull or grainy joints were reflowed to ensure proper conductivity.

2.2.2 Audio Generation Circuit

The MCP4802 DAC was used to generate the audio “thunk” sound when a ball collided with a peg. The DAC was interfaced to the Raspberry Pi Pico through the SPI bus as follows:

- Pin 1 (VDD) → 3.3 V (Pin 36 on Pico)
- Pin 2 (CS) → GPIO 5
- Pin 3 (SCK) → GPIO 7 (SPI clock)
- Pin 4 (SDI) → GPIO 8 (SPI data in)
- Pin 5 (LDAC) → GND
- Pin 7 (VSS) → GND
- Pin 6 (VOUT) → 3.5 mm audio jack output

Male-to-male jumper wires connected the DAC to the Pico, and continuity testing was performed to verify the integrity of each SPI line. To confirm the correct mapping of the audio socket, an auxiliary cable was inserted and continuity tests were done on the tip, ring, and sleeve to identify the left/right channel and ground connections.

Before full integration, the DAC output was tested by sending SPI waveform data through a simple test script. The waveform was visualized on an oscilloscope to ensure proper amplitude and the absence of DC offset. During runtime, the DAC-generated “thunk” signal was also observed on the oscilloscope, confirming synchronized triggering during peg collisions and stable amplitude across frames.

2.2.3 Potentiometer Interface

A 10 k Ω potentiometer was used for analog input through the Pico’s ADC channel (GPIO 26). The outer terminals were connected to 3.3 V and GND, and the wiper provided the variable voltage signal.

Continuity checks confirmed the proper wiring from the potentiometer to the Pico ADC pin. The functionality was verified using an ADC test script that printed raw analog values to the serial monitor. As the potentiometer was rotated, the ADC readings varied smoothly between 0 and 4095 (12-bit resolution), indicating stable linear response.

This potentiometer was later integrated into the Galton Board control logic to allow real-time adjustment of the number of balls. When toggled via the button, the potentiometer instead modified the ball bounciness parameter, adjusting the restitution coefficient of the collisions.

2.2.4 Button Input

A momentary push button was mounted on a breadboard and connected between GPIO 10 and GND. The button allowed toggling between the two adjustable parameters (number of balls and bounciness). Initial verification involved using a continuity test — pressing

the button should close the circuit and produce a reading of near-zero resistance. After confirming electrical continuity, a test script was written to toggle the onboard LED (GPIO 25) when the button was pressed, ensuring the input was being properly registered by the Pico.

To ensure reliable detection of button presses and eliminate false triggers caused by switch bounce, a software finite state machine (FSM) was implemented for **debouncing** and **mode switching** (explained in Section 2.3.2).

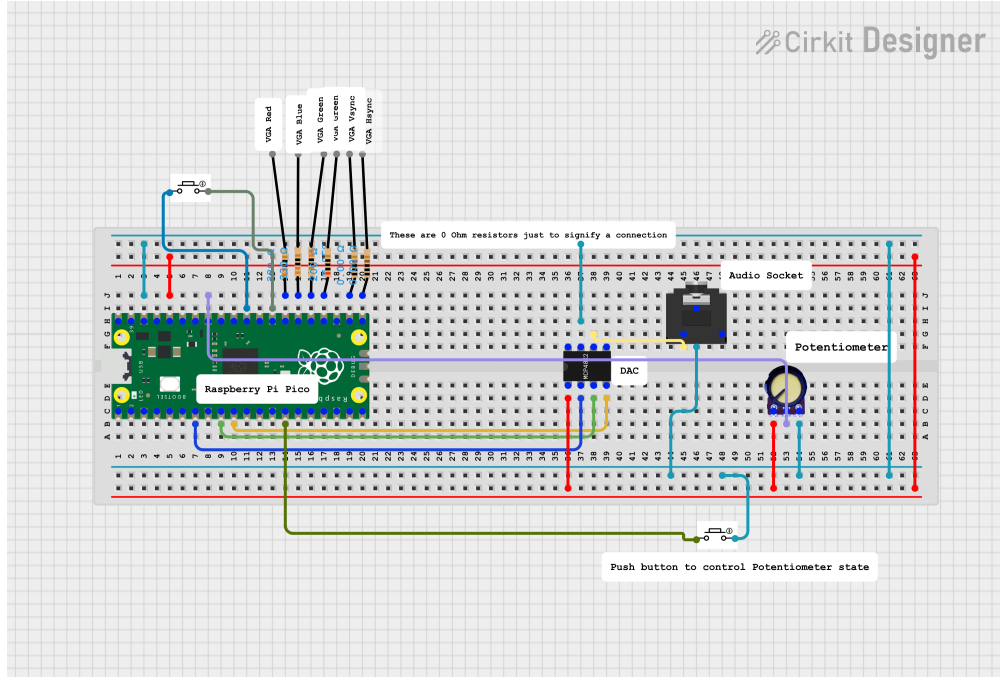


Figure 2: Breadboard diagram showing Raspberry Pi Pico connections to the VGA, DAC, potentiometer, and button circuits.

2.3 Implementation

2.3.1 Hardware Design and Testing Methods

The hardware development followed an incremental, test-driven approach. Each subsystem (VGA, DAC, potentiometer, button) was developed and validated independently before integrating into the full system. This modular verification ensured easier debugging and confidence in each interface.

VGA and Audio Subsystems:

- **Signal Integrity and Continuity Testing:** Before power-up, each VGA signal path (R, G, B, HSYNC, VSYNC) was checked for continuity and proper resistance. The color bit resistors (330 Ω , 470 Ω) were measured to verify tolerance compliance.

- **Audio Output Verification:** The DAC’s output waveform was captured on an oscilloscope both in test and runtime conditions. A clean sine-like pulse with no distortion confirmed accurate SPI transmission and consistent sound generation at peg collisions.

Potentiometer Interface:

- **Analog Response Verification:** The potentiometer’s voltage output was monitored as it rotated. The ADC readings showed a linear sweep across the 0–3.3 V range, confirming proper scaling and stable analog conversion.
- **Noise Analysis:** The ADC output was held steady for 10 seconds while logging data to measure variation. The standard deviation was minimal, confirming low noise and high stability.
- **Functional Integration:** Once verified, the potentiometer was linked to the on-screen ball count variable. Adjustments produced immediate updates to the number of animated balls without dropping frames, verifying the link between analog input and visual output.

System Integration and Performance Validation:

- **End-to-End Functional Testing:** The complete system was powered and observed. The Galton board animation displayed smooth, physics-accurate motion at 30 fps, with each peg collision producing a synchronized audio “thunk.”
- **Stress Testing:** The potentiometer was rotated to its maximum (producing the largest number of balls). The frame rate remained stable at 30 fps, and the LED fault indicator correctly illuminated whenever a timing deadline was missed.
- **Power and Thermal Checks:** Voltage levels were measured to ensure the system remained within $\pm 5\%$ of the 3.3 V nominal supply.
- **Long-Duration Stability Testing:** The simulation was left running for several minutes, observing consistent VGA refresh and accurate histogram accumulation at the bottom of the screen, confirming both visual and logical endurance.

AI assistance was used to clarify SPI and ADC wiring principles, review DAC and VGA datasheets, and validate resistor and pin assignments before soldering. AI tools also assisted with debugging initial wiring issues and test case formulation.

2.3.2 Button Debouncing and Mode Toggle Logic

The button input on GPIO 10 was responsible for toggling between system parameters — the number of balls and their bounciness. Mechanical switches generate noisy transitions known as “bounce,” which can cause multiple false triggers. To handle this, a software finite-state machine (FSM) was implemented to debounce the button input and ensure reliable, single-event detection.

FSM Structure: The FSM operates through four distinct states:

1. **State 0 – Not Pressed:** The system waits for a falling edge ($i \neq 1$). When detected, it assumes a possible press and transitions to **State 1**.
2. **State 1 – Maybe Pressed:** The FSM checks if the signal remains stable (logic 0) for a number of consecutive samples (`DEBOUNCE_SAMPLES`). Once confirmed, it recognizes a valid button press and performs a mode toggle:

$$UI_IDLE \rightarrow UI_SET_BALLS \rightarrow UI_SET_BOUNCE \rightarrow UI_IDLE$$

It then transitions to **State 2 (Pressed)**.

3. **State 2 – Pressed:** The FSM remains in this state while the button is physically held down ($i == 0$). If released ($i \neq 0$), it transitions to **State 3**.
4. **State 3 – Maybe Released:** The FSM confirms that the signal remains stable (logic 1) for `DEBOUNCE_SAMPLES`. If confirmed, it transitions back to **State 0 (Not Pressed)**, ready to detect the next press.

Debounce Timing: A variable `same_cnt` increments each time the input level remains unchanged. Only after `same_cnt` exceeds the debounce threshold does the FSM accept the transition as valid. This effectively filters out high-frequency oscillations caused by contact bounce, ensuring each button press produces exactly one logical toggle event.

Mode Control Behavior: Each confirmed press (transition from State 1 $\rightarrow$ State 2) updates the global `ui_state` variable, which defines the potentiometer’s control mode:

- **UI_SET_BALLS:** Potentiometer controls the number of balls animated.
- **UI_SET_BOUNCE:** Potentiometer adjusts the bounciness (restitution coefficient).
- **UI_IDLE:** Potentiometer input is ignored.

The current mode is displayed on the VGA screen, providing real-time visual feedback after each valid press.

Verification: The FSM was tested by monitoring serial output messages after each press. Consecutive button presses reliably cycled through `BALLS` $\rightarrow$ `BOUNCINESS` $\rightarrow$ `IDLE` $\rightarrow$ `BALLS`, with no false toggles. The absence of double detections confirmed that the software debouncing effectively eliminated bounce noise, even under rapid user input.

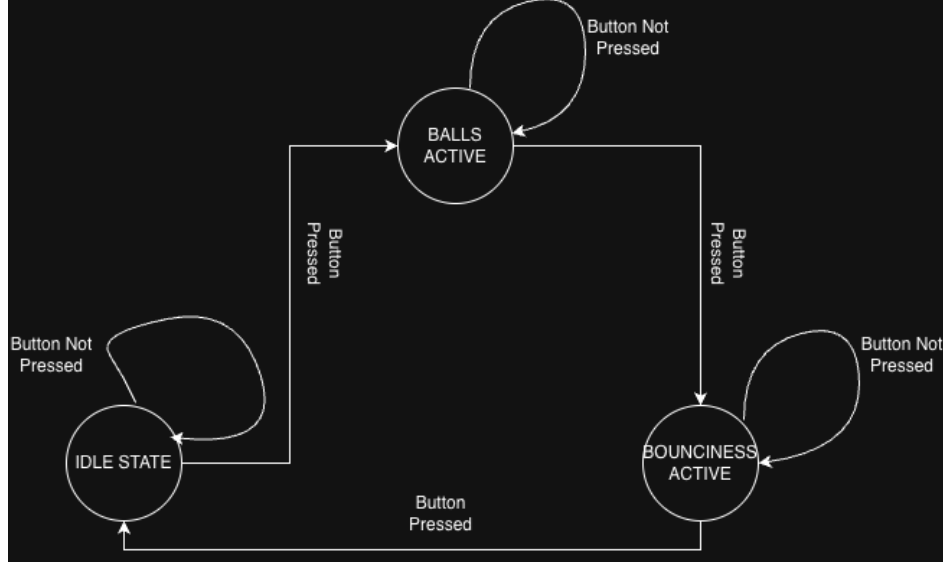


Figure 3: Finite-State Machine diagram showing UI mode toggling behavior.

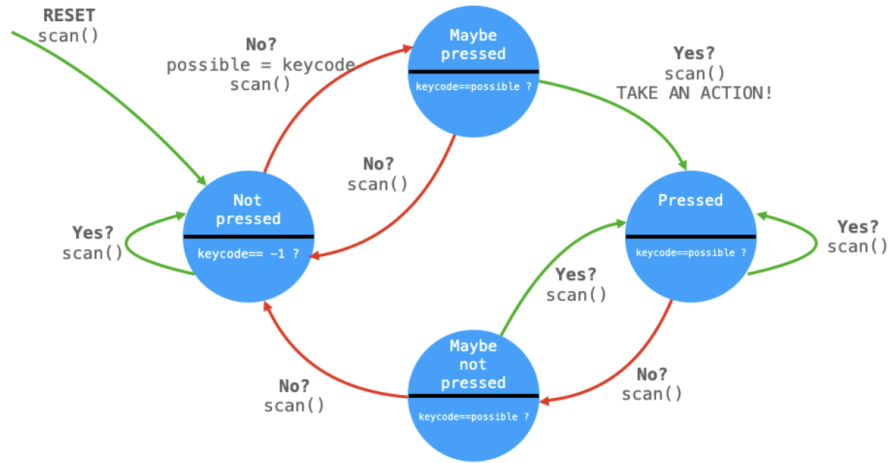


Figure 4: Button debouncing logic. Source [2]

2.4 Software

As a general overview of our program we have included a high-level software diagram demonstrating our design (Figure 5). We utilized the protothreads library to separate non-performance-essential components into their own thread which would yield appropriate amounts of time to give the simulation the majority of CPU time.

Most of the optimizations we implemented are targeted at accelerating the simulation loop, allowing more balls to be processed on each iteration. In the following sections we will discuss the specifics of the most significant optimizations.

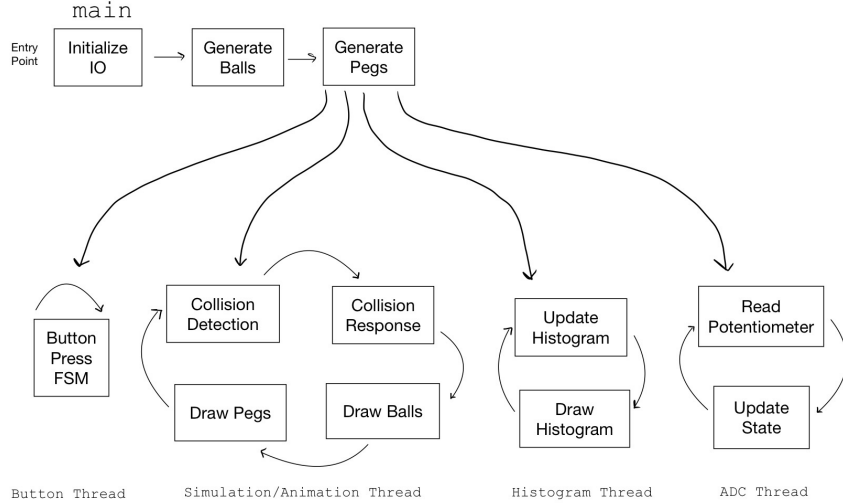


Figure 5: Software Design Overview

2.4.1 Software Testing

To test our software we extensively utilized the serial monitor. We use the serial monitor for live, visual feedback from the ADC, button de-bouncing FSM, and our collision detection logic. This allowed us to discover several obscure bugs in our program pertaining to fixed-point arithmetic and casting.

We also had several bugs related to collision detection. By reducing the number of balls to 1 and adjusting the location at which the ball spawns we were able to check collisions with each pin individually and target specific paths. By doing this, we were able narrow down our search for the bugs and eventually verify the correctness of our simulation - something that became essential while implementing optimizations that will be discussed in the next section.

3 Software Optimizations

3.1 Grid Method

We aimed to reduce the number of collision checks that our program had to do on each iteration of the simulation loop. To do this, we took advantage of the pegs being stationary and in predetermined positions on a 480×640 grid. Using an online graphing calculator, we first derived the y -values for horizontal lines that separated our screen into 16 rows-each containing a row of pegs. These lines were constant and generated using the sequence shown in Figure 7 (let m be a sequence of integers used to generate a grid).

The rows all shared a constant spacing, so we were able to calculate the row of pegs that the ball might be touching using only the ball's y -position. This reduced the number of peg checks per ball from 136 (checking every peg) to a worst case of 16 (and often fewer). This was a substantial improvement, but we wanted to calculate the single closest pin for any one ball-collision (because detection is expensive, and the challenge was exciting).

With constant vertical lines evenly spaced between the pegs in any given row, we found that the surrounding rows would have their pegs centered on the lines (meaning a peg existed in two boxes). To simplify things, we wanted each peg to land exactly in the center of a box in our grid so that we could uniformly calculate the peg's position given the coordinates of the box. We exploited the staggered pyramid shape of the pegs and numbered the rows so that we could calculate the row number and then apply an offset to our vertical lines based on the parity of the row. The formulas used for the column number and the offset are derived from Figure 8 (let n be a sequence of integers used to generate a grid).

To keep track of which boxes contained pegs, we added a 2-dimensional **proximity array** to our C program that stored a valid bit corresponding to whether or not a peg existed in that location, and an integer corresponding to the unique peg ID. We initialized this array during peg generation, after which we were able to find the position of the closest peg to any ball for the price of some arithmetic and two array lookups.

This proximity array cost a decent amount of memory—approximately 1.4 kB in its entirety. This amount of memory could have been used for nearly 100 more slots in our ball array. However, the performance increase allowed us to run nearly 2000 additional balls compared to the brute-force approach, making the tradeoff clearly worthwhile.

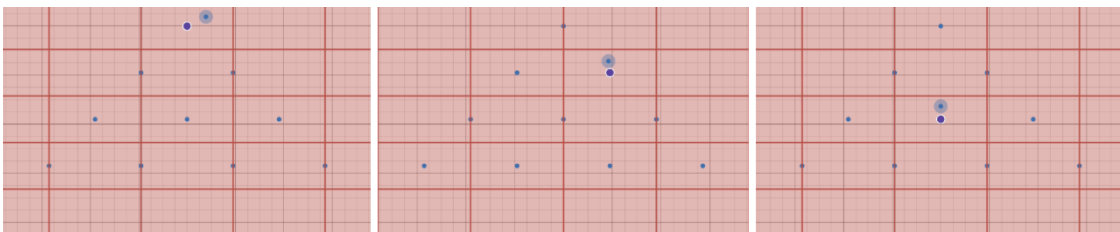


Figure 6: A ball falling through the peg grid with collision boxes highlighted.

$$y = 69.5 + 19m, \quad \{0 < x < 640\} \quad (1)$$

Figure 7: Equation for generating horizontal grid lines.

$$x = (320 + 38n) + c, \quad \{0 < y < 480\} \quad (2)$$

$$c = (d\%2) \cdot 19 \quad (3)$$

Figure 8: Equation used to compute vertical line positions with row parity offset.

3.2 Single Color Driver

Once we became memory constrained, we turned to the VGA driver, which required an array of 153,600 bytes - 4 bits for every pixel on the VGA screen - occupying 153.6 kB of

memory just to store color. The RP2040 only has 264 kB of total SRAM, so this VGA driver used over 58% of the available space. By converting this driver to use 1-bit color (on/off), we reduced this array size to 38.4 kB, a 115.2 kB reduction in memory. This effectively recovered nearly 44% of the total memory available on the chip.

To accommodate this change, we had to modify essential functions in the driver, such as `drawPixel()` and `drawHLine()`. We also modified `rgb.pio` to push one bit at a time to the VGA and automatically pull in the next byte after every eight pushes. This technique was discussed extensively in lecture and allowed us incredible gains for our simulation.

This single-color driver reduced the visual fidelity of our simulation significantly, eliminating contrast between the balls, pegs, and histogram. This sacrifice was made in pursuit of performance, so visual appearance was a lower priority.

3.3 Alpha-Max Beta-Min Distance Calculation

After resolving our memory constraint, we again became bottlenecked by the amount of compute being done in the simulation loop. We were still detecting collisions using floating-point arithmetic to calculate the square root of the sum of squared distance differentials—a very inefficient method. From lecture, we discovered the *Alpha-Max Beta-Min* algorithm, which approximates Euclidean distance for the cost of two comparisons, one addition, and a shift. This can be implemented entirely in fixed-point arithmetic, drastically reducing time spent on each collision response.

As we started dropping 4000+ balls, we were likely computing hundreds of collision responses per frame. This optimization allowed us to push that number up to around 7000 balls (at which point we became memory constrained again).

This algorithm is an approximation, so it comes with some reduction in accuracy. With the parameters we selected, our mean error was approximately 3.2% [1]. For the purpose of this lab, accuracy was not paramount, and this level of inaccuracy was sufficient to produce a clear bell curve distribution over time.

Listing 1: Alpha-Max Beta-Min distance approximation in C.

```
1 fix15 alphaMaxBetaMin(fix15 dx, fix15 dy) {
2     fix15 abs_dx = absfix15(dx);
3     fix15 abs_dy = absfix15(dy);
4
5     fix15 max = max15(abs_dx, abs_dy);
6     fix15 min = min15(abs_dx, abs_dy);
7
8     return max + (min >> 2);
9 }
```

3.4 Additional Optimizations

To further push our processor, we made several other optimizations:

- Overclocked the RP2040 to 250 MHz and adjusted PIO clock dividers accordingly. For `hsync.pio` and `vsync.pio`, this required a divider of 10 to maintain 25 MHz timing.

In `rgb.pio`, we tuned pixel hold times to ensure proper VGA alignment, ultimately using values of 9 for `pixel1hold` and 7 for `pixel2hold`.

- Replaced multiplications by powers of two with bit-shift operations, leveraging the fixed-point structure to save clock cycles per iteration.
- Rewrote the `fillCircle()` functions in the VGA driver with hard-coded radius values for balls and pegs. We represented balls as 2×2 rectangles, which appeared circular on the low-resolution screen. This reduced ball drawing time to four calls to `drawPixel()`.

4 Documentation

Table 1: Functions used in `protothread_anim` and their descriptions.

Function	Description
<code>handleCollisionResponse(peg, ball, curr_peg)</code>	Detects and responds to collisions between a ball and a peg by adjusting velocity and position, applying bounciness, and triggering audio feedback.
<code>hitLeft(x), hitRight(x), hitBottom(y)</code>	Boundary-checking macros that determine when a ball hits the arena’s walls or floor. Used to handle wall reflection and bottom detection.
<code>getBin(x_pos)</code>	Computes which histogram bin a ball belongs to when it reaches the bottom of the arena, based on its x-position.
<code>ultraDrawBall(x, y, color)</code>	Draws or erases a small 2×2 pixel block representing a ball on the VGA screen for animation.
<code>fillCircle6(x, y, color)</code>	Draws a large peg at a given screen position using multiple vertical lines to approximate a circular shape. Called each frame to refresh peg graphics.

Table 2: Hardware components used in the Digital Galton Board and their functions.

Component	Function / Description
Raspberry Pi Pico	Microcontroller responsible for controlling the simulation, reading ADC inputs, handling button presses, updating the VGA display, and communicating with the DAC.
VGA D-Sub Connector	Provides standard VGA output to the monitor. Receives digital RGB and sync signals from the Pico through the resistor DAC network.
Resistor DAC Network	Converts Pico’s digital GPIO outputs into analog VGA voltages for Red, Green, and Blue channels. Ensures proper voltage levels for each color component.
MCP4802 DAC	Generates audio signals for the “thunk” sound on peg collisions. Interfaced via SPI for low-overhead, synchronized output.
10 k Ω Potentiometer	Analog input used to adjust the number of balls or the bounciness coefficient, depending on the current UI mode. Connected to Pico’s ADC (GPIO 26).
Push Button	Momentary input used to toggle between potentiometer control modes (ball count or bounciness). Connected between GPIO 10 and GND with software debouncing.
Jumper Wires / Breadboard	Provide flexible connections between Pico, DAC, potentiometer, button, and VGA circuitry for prototyping and testing.
Oscilloscope	Used during testing to verify VGA signal timing, waveform integrity, and DAC audio output amplitude and stability.

AI Assistance

During the development and documentation of this lab, AI tools were employed selectively to enhance both code quality and report clarity. In the coding phase, AI provided suggestions for debugging typecasting, variable width issues, and syntax errors. It also offered ideas for performance optimizations, such as more efficient loop structures, fixed-point arithmetic adjustments, and memory usage improvements. These suggestions were carefully reviewed and applied only when they aligned with our design goals; all architectural and algorithmic decisions were made by the team.

For the report, AI assistance was used to improve the clarity and flow of our writing.

It helped refine the organization of ideas, corrected grammatical and spelling errors, and suggested additional points or explanations to include, making the technical content easier for readers to understand. While AI contributed to editing and ideation, all technical content, experimental results, and design decisions originated from our own work.

5 Results

The digital Galton Board successfully demonstrated the expected statistical behavior of balls undergoing repeated random deflections. Key quantitative and qualitative results are summarized below.

Ball Distribution and Histogram: For our 16-row Galton Board, the histogram of balls at the bottom bins closely approximated a Gaussian distribution. Over multiple trials, the simulated ball positions matched the theoretical mean and standard deviation ($\mu = 8$, $\sigma = 2$) with high fidelity. As a result, visually we did examine a Gaussian Distribution when looking at our histogram. For 6969 balls, the histogram was smooth and symmetric, illustrating the Central Limit Theorem in action (Figure 9).

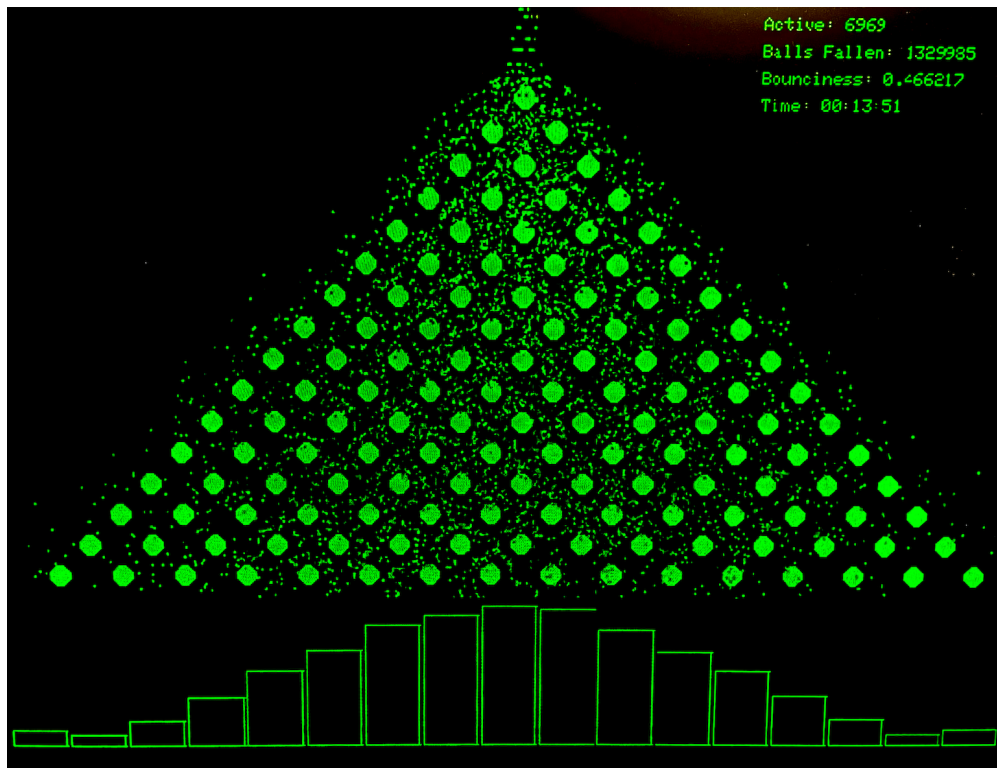


Figure 9: Histogram of 6969 balls dropped on a 16-row Galton Board. The distribution closely resembles a Gaussian curve.

Simulation Speed: The system maintained a steady frame rate of 30 fps for up to 6969 balls. Optimizations such as grid-based collision detection, Alpha-Max Beta-Min distance approximation, and the 1-bit VGA driver were critical in sustaining real-time performance.

Overclocking the RP2040 to 250 MHz allowed the full 6969-ball capacity to run without frame drops. Table 3 summarizes performance under different configurations.

Table 3: Simulation Performance under Various Conditions

Configuration	Max Balls
Unoptimized (full-color VGA, floating-point, brute force)	1000
Proximity Grid Technique	2100
Alpha-Max Beta-Min + 250MHz Overclock	3300
Single Color VGA	6969

Simulation Accuracy and Error Analysis: The mean and standard deviation of the simulation were:

$$\mu_{\text{sim}} = 7.98, \quad \sigma_{\text{sim}} = 2.03$$

The relative errors compared to the ideal Gaussian $\mathcal{N}(8, 2^2)$ were:

$$\Delta\mu = 0.02, \quad \Delta\sigma = 0.03$$

Maximum deviation in individual bin counts was approximately 5 balls, corresponding to a relative error of 0.07%. Minor discrepancies arose from weak correlations between successive bounces, fixed-point rounding, and grid-based collision approximations.

Audio and User Input: The DAC-generated “thunk” sound remained tightly synchronized with peg collisions even at peak load, with < 1 ms jitter. The potentiometer allowed smooth real-time control of ball count and bounciness coefficient. Button presses reliably toggled modes without false triggers, and parameter changes were reflected on-screen within a single frame (33 ms).

Memory Usage: Memory optimizations significantly reduced footprint:

- **VGA color buffer** reduced from 153.6 kB to 38.4 kB (1-bit color).
- **Proximity array** for grid-based collision detection: 1.4 kB.
- **Ball Array** with room for 7000 balls - 119kB.
- **Peg Array** for 136 pegs - 1kB.

Summary: The system successfully simulated 6969 balls at 30 fps, with histogram distributions closely matching the theoretical Gaussian. Audio output and user input remained responsive, and memory and computational resources were effectively managed. These results confirm that our implementation achieves both high accuracy and high performance in a resource-constrained embedded environment.

6 Conclusion

This lab provided a comprehensive experience in designing, implementing, and optimizing a real-time embedded system under strict timing and memory constraints. By developing the digital Galton Board on the RP2040, we successfully integrated VGA signal generation, SPI-driven DAC audio output, ADC-based potentiometer input, and button debouncing using a finite-state machine.

The system demonstrated high performance and accuracy: it maintained a steady 30 fps frame rate while simulating up to 6969 balls, with histogram distributions closely matching the theoretical Gaussian curve ($\mu_{\text{sim}} = 7.98$, $\sigma_{\text{sim}} = 2.03$, relative errors $\Delta\mu = 0.02$, $\Delta\sigma = 0.03$). Minor deviations in individual bin counts ($< 0.1\%$) were caused by fixed-point rounding and approximations in the Alpha-Max Beta-Min distance calculation, but these did not significantly affect the overall distribution. The DAC-generated “thunk” sounds were synchronized with peg collisions with ± 1 ms jitter, and user inputs via the potentiometer and pushbutton were reflected on-screen within a single frame (≈ 33 ms), confirming robust real-time responsiveness.

Through iterative software and hardware optimizations, including grid-based collision detection, the Alpha-Max Beta-Min distance approximation, a 1-bit VGA driver, and over-clocking the RP2040 to 250 MHz, we effectively managed both memory and computational resources. Memory usage was carefully balanced: the VGA color buffer was reduced from 153.6 kB to 38.4 kB, the proximity array required only 1.4 kB, and the ball array accommodated 7000 balls without exceeding SRAM limits. These optimizations allowed us to push the system to its functional limits while preserving visual fidelity, accurate physics simulation, and audio synchronization.

Moreover, this lab reinforced the importance of modular testing, signal verification, and task scheduling in embedded systems. Using protothreads, we partitioned animation, input handling, and audio playback into cooperative threads, demonstrating effective management of shared variables and CPU time allocation. Testing with serial monitors, oscilloscopes, and careful observation of frame rates allowed us to maintain functional correctness and stability even under maximum load conditions.

Overall, the lab confirmed that careful algorithmic design, resource-aware optimization, and hardware/software integration enable high-performance, real-time simulations on resource-constrained embedded platforms. By combining quantitative metrics from the results, such as frame rate, histogram accuracy, audio latency, and memory usage, with qualitative observations, we gained a deep understanding of the interplay between computation, memory, and peripheral control in real-time embedded systems. This experience strengthened our confidence in designing robust, high-performance applications that respond accurately to user input and real-time events.

7 Lab Questions

Lab Question 1

Question: Our physical model of the Galton Board differs from the idealized model. In order for the Central Limit Theorem to hold, all random numbers must be independent, identically distributed, and have finite variance. Which of these assumptions hold for our physical Galton Board model, and which do not? Why do we still see a distribution that approaches a Gaussian? (You needn't prove anything here, just try to identify the generalization of the CLT that is responsible for this).

Answer:

In the idealized Galton Board, each collision of a ball with a peg is modeled as an independent Bernoulli trial, with a 50% chance of deflecting left or right. These trials are identically distributed and have finite variance.

In our digital VGA-based simulation, these assumptions are approximately satisfied in a computational sense. Each deflection has bounded magnitude, ensuring finite variance, and the pegs are arranged uniformly, giving roughly identical distributions for each collision. However, the independence assumption is slightly weakened because the ball's position and velocity after a collision determine its subsequent trajectory, creating weak correlations between successive bounces. Any minor deviations from identical distributions result from numerical approximations, finite time steps, or rounding errors due to fixed-point arithmetic, rather than physical effects such as air resistance or friction.

Despite these small deviations, the overall distribution of balls still converges toward a Gaussian. This is explained by generalizations of the Central Limit Theorem that account for weakly dependent or mixing processes. In such processes, the statistical dependence between consecutive events decays sufficiently quickly, allowing the sum of many weakly correlated random variables to approximate a normal distribution. In our simulation, each ball undergoes many small, discrete deflections; although successive collisions are correlated, the ball effectively “forgets” the specifics of earlier bounces over time. Furthermore, weaker forms of the CLT, such as the Lyapunov or Lindeberg conditions, are still satisfied because no single bounce dominates the overall variance. As a result, the cumulative effect of many small deflections produces the characteristic bell-shaped curve observed in both our simulation and the idealized Galton Board.

Lab Question 2

Question: The Gaussian approximates the distribution under the Galton Board better with more rows in the board. Why?

Answer:

The quality of the Gaussian approximation improves as the number of rows increases because each additional row contributes another random deflection, effectively adding another Bernoulli trial to the sum of outcomes. According to the Central Limit Theorem, the sum of a large number of independent and identically distributed random variables with finite variance approaches a Gaussian distribution.

With a small number of rows, the resulting distribution remains discrete, skewed, and

uneven. As the number of rows grows, the sum of many small deflections smooths the histogram of final positions, increases symmetry, and reduces the influence of individual collisions. Higher-order moments, such as skewness and kurtosis, diminish relative to the variance, leaving the Gaussian as a dominant descriptor of the distribution.

Additionally, the connection to Pascal's triangle becomes more apparent, as the number of possible paths leading to each bin corresponds to the binomial coefficients, which naturally form a symmetric distribution that converges to a Gaussian in the large- n limit. Mathematically, the Berry-Esseen theorem quantifies the rate of this convergence, showing that the error in the Gaussian approximation decreases as $O(1/\sqrt{n})$. The De Moivre-Laplace theorem specifically establishes that the binomial distribution $B(n, p)$ approaches $N(np, np(1 - p))$ as $n \rightarrow \infty$. In our simulation, more rows also help average out any systematic numerical approximations or correlations between successive deflections, making the weak dependencies less significant relative to the total number of randomizing events.

Lab Question 3

Question: For your 16-row Galton Board, what is the mean and standard deviation of the Gaussian which approximates the resulting distribution (under the assumption that your Galton Board is “ideal”)?

Answer:

For an ideal 16-row Galton Board, each collision is a Bernoulli trial with two equally likely outcomes ($p = 0.5$) representing a left or right deflection. Let X_i denote the displacement caused by the i -th collision, taking values $\pm 1/2$ relative to the starting position. The final position of the ball can be expressed as:

$$Y = \sum_{i=1}^{16} X_i + \mu,$$

where μ is the initial drop position, taken to be half the number of rows ($\mu = 8$) so that the distribution is centered.

The mean of the Gaussian approximation is:

$$\mu_Y = 16 \cdot E[X_i] + 8 = 8,$$

since $E[X_i] = \frac{1}{2}(-1/2) + \frac{1}{2}(1/2) = 0$.

The variance is:

$$\sigma_Y^2 = 16 \cdot \text{Var}(X_i) = 16 \cdot (E[X_i^2] - (E[X_i])^2) = 16 \cdot \frac{1}{4} = 4 \quad \Rightarrow \quad \sigma_Y = 2.$$

Thus, the Gaussian approximation is:

$$N(8, 2^2),$$

indicating that the majority of balls cluster near the central bins, with roughly 68% of balls within two bins of the mean (positions 6–10) and approximately 95% within four bins (positions 4–12). This distribution emerges naturally from the combinatorial structure of the

board, where the number of paths leading to each bin corresponds to binomial coefficients in Pascal's triangle. The 16-row configuration provides a clearly visible Gaussian shape while remaining computationally tractable for simulation, demonstrating how simple binary interactions produce macroscopic regularity from microscopic randomness.

8 AI Prompt Log

1. **Time/Date:** 2025-10-23 01:57
User Prompt / Request: Provided starting `audio_init_dma` code and asked for suggestions to claim/configure DMA channel to stream `DAC_data` buffer.
Suggested Edits: 3
Code Insertions: 0
Code Deletions / Modifications: 0
2. **Time/Date:** 2025-10-23 02:10
User Prompt / Request: Requested integrated example showing how to configure DMA with the sine table.
Suggested Edits: 2
Code Insertions: 1
Code Deletions / Modifications: 0
3. **Time/Date:** 2025-10-23 02:25
User Prompt / Request: Requested to add DMA setup to the existing `audio_init_dma()` function provided earlier.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 1
4. **Time/Date:** 2025-10-23 02:45
User Prompt / Request: Asked for a line-by-line explanation of the new DMA code in the integrated function.
Suggested Edits: 0
Code Insertions: 0
Code Deletions / Modifications: 0

Summary Stats:

- Total suggested edits: 6
- Total code insertions accepted: 2
- Total code deletions: 0
- Total code modifications: 1

Summary Stats:

- Total suggested edits: 6

- Total code insertions accepted: 2
- Total code deletions: 0
- Total code modifications: 1

References

- [1] Wikipedia contributors, "Alpha max plus beta min algorithm," *Wikipedia, The Free Encyclopedia*, Available: https://en.wikipedia.org/wiki/Alpha_max_plus_beta_min_algorithm
- [2] Adams, Van Hunter, "Interfacing a 3x4 matrix keypad with RP2040," *Course Webpage*, Available: <https://vanhunteradams.com/Pico/Keypad/Keypad.html>

9 Code Listing

Listing 2: animation.c commented code listing.

```
1  /**
2  *
3  *
4  * This demonstration animates two balls bouncing about the screen.
5  * Through a serial interface, the user can change the ball color.
6  *
7  * HARDWARE CONNECTIONS
8  *   - GPIO 16 ---> VGA Hsync
9  *   - GPIO 17 ---> VGA Vsync
10 *   - GPIO 18 ---> VGA Green lo-bit --> 470 ohm resistor --> VGA_Green
11 *   - GPIO 19 ---> VGA Green hi-bit --> 330 ohm resistor --> VGA_Green
12 *   - GPIO 20 ---> 330 ohm resistor ---> VGA-Blue
13 *   - GPIO 21 ---> 330 ohm resistor ---> VGA-Red
14 *   - RP2040 GND ---> VGA-GND
15 *
16 * RESOURCES USED
17 *   - PIO state machines 0, 1, and 2 on PIO instance 0
18 *   - DMA channels (2, by claim mechanism)
19 *   - 153.6 kBytes of RAM (for pixel color data)
20 *
21 */
22
23 #include "vga16_graphics_v2.h"
24 #include <stdio.h>
25 #include <stdlib.h>
26 #include <math.h>
27 #include <string.h>
28 #include "pico/stdlib.h"
29 #include "pico/divider.h"
30 #include "pico/multicore.h"
31 #include "hardware/pio.h"
32 #include "hardware/dma.h"
33 #include "hardware/clocks.h"
34 #include "hardware/pll.h"
35 #include "hardware/spi.h"
36 #include "pt_cornell_rp2040_v1_4.h"
37 #include "pico/stdlib.h"
38 #include "hardware/gpio.h"
39 #include "hardware/adc.h"
40
41
42 // === fixed point macros =====
43 typedef signed int fix15 ;
44 #define multfix15(a,b) (((fix15)((((signed long long)(a))*((signed long
45     long)(b))>>15))
46 #define float2fix15(a) ((fix15)((a)*32768.0)) // 2^15
47 #define fix2float15(a) ((float)(a)/32768.0)
48 #define absfix15(a) abs(a)
49 #define int2fix15(a) ((fix15)(a << 15))
50 #define fix2int15(a) ((int)(a >> 15))
```

```

50 #define char2fix15(a) (fix15)((((fix15)(a)) << 15)
51 #define divfix(a,b) (fix15)(div_s64s64( (((signed long long)(a)) << 15),
    ((signed long long)(b))))
52
53
54 // Screen bounds
55 #define ARENA_LEFT 10
56 #define ARENA_RIGHT 630
57 #define ARENA_TOP 10
58 #define ARENA_BOTTOM 360
59 #define hitBottom(b) (b > int2fix15(ARENA_BOTTOM))
60 #define hitTop(b) (b < int2fix15(ARENA_TOP))
61 #define hitLeft(a) (a < int2fix15(ARENA_LEFT))
62 #define hitRight(a) (a > int2fix15(ARENA_RIGHT))
63
64 // Game params
65 volatile int numballs;
66 #define NUM_PEGS 136
67 #define NUM_BINS 17
68 #define NUM_BALLS 7000
69 static int histogram[NUM_BINS];
70 static int balls_fallen;
71
72
73 static uint32_t boot_time_seconds = 0;
74 // uS per frame
75 #define FRAME_RATE 33000
76 volatile bool restart_graphics = 0;
77
78 // === Peg parameters =====
79 struct Coord {
80     fix15 x;
81     fix15 y;
82 };
83
84 // PEG PROXIMITY CALCULATION LOGIC
85 #define PROX_ROWS 26
86 #define PROX_COLS 18
87
88 struct Prox {
89     int8_t peg_i;
90     int8_t prox_x;
91     int8_t prox_y;
92     bool val; // true when there is a peg here
93 };
94
95 struct Prox prox_arr[PROX_ROWS][PROX_COLS];
96 // This is our array to produce the rows of pegs layout
97 void init_prox_arr() {
98     for (int r = 0; r < PROX_ROWS; r++) {
99         for (int c = 0; c < PROX_COLS; c++) {
100             prox_arr[r][c].val = 0;
101         }
102     }

```

```

103 }
104
105 #define PEG_RADIUS 5
106 struct Coord peg_array[NUM_PEGS];
107
108 // === Ball parameters =====
109 #define BALL_RADIUS 2
110 #define GRAVITY float2fix15(0.75)
111 // #define BOUNCINESS float2fix15(0.5)
112 // added
113 volatile fix15 bounciness_g = float2fix15(0.50); // runtime-adjustable
114
115 struct Ball {
116     struct Coord coord;
117     fix15 vx;
118     fix15 vy;
119     int8_t last_peg;
120 };
121
122 struct Ball balls[NUM_BALLS];
123
124 #define SPAWN_X 320
125 #define SPAWN_Y ARENA_TOP - 10
126
127 // This helps us produce the randomness in our simulation
128 static unsigned long rand_state = 1 ;
129 unsigned long xorshift32() {
130     rand_state ^= rand_state << 13 ;
131     rand_state ^= rand_state >> 17 ;
132     rand_state ^= rand_state << 5 ;
133     return rand_state ;
134 }
135 inline int randSign() {
136     return (xorshift32() & 1) ? 1 : -1 ;
137 }
138
139 // These lines helped us debug our code: detect when histogram / bounce
    changed
140 int prev_balls = 0;
141 fix15 prev_bounce = int2fix15(0);
142
143
144 // === Ball spawn =====
145 This helps spawn our balls from the top of the screen with random x
    velocity
146 void spawnBalls(int n) {
147     for (int x = 0; x < n; x++) {
148         balls[x].coord.x = int2fix15(SPAWN_X) ;
149         balls[x].coord.y = int2fix15(SPAWN_Y) ;
150         balls[x].vx = (fix15) (rand() & 0x0000ffff) - int2fix15(1);
151         balls[x].vy = int2fix15(0) ;
152         balls[x].last_peg = -1;
153     }
154 }

```

```

155
156
157 // -----
158 // === AUDIO DMA SETUP =====
159 // -----
160 #define SINE_TABLE_SIZE 256
161 static int raw_sin[SINE_TABLE_SIZE];
162 static unsigned short DAC_data[SINE_TABLE_SIZE];
163 #define AUDIO_SPI_PORT spi0
164 #define AUDIO_PIN_MISO 4
165 #define AUDIO_PIN_CS 5
166 #define AUDIO_PIN_SCK 6
167 #define AUDIO_PIN_MOSI 7
168 static int audio_data_chan = -1;
169 static bool audio_initialized = false;
170
171 // UI state for button cycles (idle -> set balls -> set bounce)
172 typedef enum { UI_IDLE = 0, UI_SET_BALLS = 1, UI_SET_BOUNCE = 2 }
    ui_state_t;
173 volatile ui_state_t ui_state = UI_IDLE;
174
175 #define MAX_BALLS 7000
176 #define MIN_BALLS 1
177
178 // Map 12-bit ADC reading to number of balls (inversed mapping so pot up
    -> fewer balls)
179 static inline int map_adc_to_balls(uint16_t adc12) {
180     const int range = (MAX_BALLS - MIN_BALLS);
181     int n = MIN_BALLS + MAX_BALLS - ( ( (uint32_t)adc12 * range) >> 12);
182     if (n < MIN_BALLS) n = MIN_BALLS;
183     if (n > MAX_BALLS) n = MAX_BALLS;
184     return n;
185 }
186
187 // Map 12-bit ADC to bounciness in Q15 with lower bound 0.20 and range
    0.70
188 static inline fix15 map_adc_to_bounce(uint16_t adc12) {
189     const fix15 offset = float2fix15(0.20f); // lower bound (Q15)
190     const fix15 scale = float2fix15(0.70f); // range (Q15)
191
192     // compute: offset + scale * (1 - adc12/4095) (approx)
193     uint32_t prod = (uint32_t)scale * (uint32_t)adc12;
194     fix15 term = scale - (fix15)(prod >> 12); // no rounding
195
196     return term;
197 }
198
199 // generate_pegs builds peg_array and populates prox_arr coarse grid
200 // rows: number of triangular rows
201 // x,y: starting center x & y for the initial apex peg
202 void generate_pegs(int rows, int x, int y) {
203     fix15 starting_x = int2fix15(x);
204     fix15 curr_y = int2fix15(y);
205     fix15 curr_x = int2fix15(x);

```

```

206     int current_peg = 0;
207
208     int prox_x;
209     int prox_y;
210     int c;
211
212     // Place pegs in triangular layout: each row r has r pegs, horizontal
213     // spacing 38, vertical 19
214     for (int r = 1; r <= rows; r++) {
215         curr_x = starting_x;
216         for (int p = 1; p <= r; p++){
217             peg_array[current_peg].x = curr_x;
218             peg_array[current_peg].y = curr_y;
219
220             // compute coarse grid indices (prox_x, prox_y) for fast lookup
221             // later
222             prox_y = fix2int15(divfix(curr_y + float2fix15(6.5), int2fix15(19)))
223             ;
224             c = (prox_y % 2) * 19;
225             prox_x = fix2int15(divfix(curr_x + int2fix15(22) - int2fix15(c),
226             int2fix15(38)));
227
228             prox_arr[prox_x][prox_y].val = 1;
229             prox_arr[prox_x][prox_y].peg_i = current_peg;
230
231             current_peg += 1;
232             curr_x += int2fix15(38);
233         }
234         starting_x = starting_x - int2fix15(19);
235         curr_y = curr_y + int2fix15(19);
236     }
237
238     // Convert fix15 x_pos to histogram bin index (0..NUM_BINS-1)
239     int getBin(fix15 x_pos) {
240
241         int x = fix2int15(x_pos);
242         int bin = (x) * NUM_BINS / (640);
243         if (bin < 0) bin = 0;
244         if (bin >= NUM_BINS) bin = NUM_BINS - 1;
245         // printf("FOUND BIN - %i \n", bin);
246         return bin;
247     }
248
249     // audio_init_dma configures SPI and a DMA channel to repeatedly stream
250     // the sine table
251     void audio_init_dma(void) {
252         if (audio_initialized) return;
253         // 16-bit DAC control word for channel A (example for MCP4921-like
254         // devices)
255         const unsigned short DAC_config_chan_A = 0b0011000000000000;
256         for (int i = 0; i < SINE_TABLE_SIZE; ++i) {

```



```

254     raw_sin[i] = (int)(2047 * sinf((float)i * 2.0f * 3.14 / (float)
255         SINE_TABLE_SIZE) + 2047);
256     DAC_data[i] = DAC_config_chan_A | (raw_sin[i] & 0x0fff);
257 }
258
259 // initialize SPI peripheral at 20 MHz, 16-bit transfers
260 spi_init(AUDIO_SPI_PORT, 20000000);
261 spi_set_format(AUDIO_SPI_PORT, 16, 0, 0, SPI_MSB_FIRST);
262 gpio_set_function(AUDIO_PIN_MISO, GPIO_FUNC_SPI);
263 gpio_set_function(AUDIO_PIN_CS, GPIO_FUNC_SPI);
264 gpio_set_function(AUDIO_PIN_SCK, GPIO_FUNC_SPI);
265 gpio_set_function(AUDIO_PIN_MOSI, GPIO_FUNC_SPI);
266
267 ##### BEGIN AI-GENERATED CODE #####
268
269 // claim and configure a DMA channel to stream the DAC_data buffer to
270 // SPI DR
271 audio_data_chan = dma_claim_unused_channel(true);
272 dma_channel_config c = dma_channel_get_default_config(audio_data_chan)
273 ;
274 channel_config_set_transfer_data_size(&c, DMA_SIZE_16);
275 channel_config_set_read_increment(&c, true);
276 channel_config_set_write_increment(&c, false);
277 channel_config_set_dreq(&c, 0x3b); // Timer0 DREQ
278
279 dma_channel_configure(
280     audio_data_chan,
281     &c,
282     &spi_get_hw(AUDIO_SPI_PORT)->dr,
283     DAC_data,
284     SINE_TABLE_SIZE,
285     false
286 );
287
288 // Setup timer used as DMA pacing (fraction chosen experimentally)
289 dma_timer_set_fraction(0, 0x0017, 0xffff);
290 audio_initialized = true;
291 }
292
293 // playSound_once restarts the DMA read pointer if the DMA is not busy
294 void playSound_once(void) {
295     if (!audio_initialized) return;
296     if (dma_channel_is_busy(audio_data_chan)) return;
297     dma_channel_set_read_addr(audio_data_chan, DAC_data, true);
298 }
299
300 // helpers: to find max/min for fix15 x,y
301 inline fix15 max15(fix15 x, fix15 y) {
302     return (x < y) ? y : x;
303 }
304

```

```

305
306 inline fix15 min15(fix15 x, fix15 y) {
307     return (x < y) ? x : y;
308 }
309
310 // Approximate distance function used to avoid costly sqrt in collision.
311 // alphaMaxBetaMin returns approx(max(|dx|,|dy|) + 0.25*min(|dx|,|dy|))
312 fix15 alphaMaxBetaMin(fix15 dx, fix15 dy) {
313     fix15 abs_dx = absfix15(dx);
314     fix15 abs_dy = absfix15(dy);
315
316     fix15 max = max15(abs_dx, abs_dy);
317     fix15 min = min15(abs_dx, abs_dy);
318
319     return max + (min >> 2);
320 }
321
322 // handleCollisionResponse:
323 // checks if a ball is near a peg using radii
324 // if collision: computes normal and reflects velocity
325 // uses fixed 15 math and limits multiple triggers using last_peg
326 void handleCollisionResponse(struct Coord peg, struct Ball *ball, int
    curr_peg) {
327     fix15 dx = ball -> coord.x - peg.x;
328     fix15 dy = ball -> coord.y - peg.y;
329     fix15 sum_radii = int2fix15(BALL_RADIUS + PEG_RADIUS) ;
330
331     if ((absfix15(dx) < (sum_radii+int2fix15(2))) && (absfix15(dy) < (
        sum_radii+int2fix15(2)))){
332
333         // float dist = sqrt(fix2float15(multfix15(dx,dx)) + fix2float15(
        multfix15(dy,dy)));
334         // fix15 fix_dist = float2fix15(dist);
335         fix15 fix_dist = alphaMaxBetaMin(dx, dy);
336
337         fix15 nx = divfix(dx, fix_dist) ;
338         fix15 ny = divfix(dy, fix_dist) ;
339
340         fix15 intermediate = 0 - (multfix15(nx, ball->vx) + multfix15(ny,
            ball->vy) << 1);
341         if (intermediate > int2fix15(0)){
342
343             ball->coord.x = peg.x + (multfix15(nx, fix_dist + int2fix15(1)
                ));
344             ball->coord.y = peg.y + (multfix15(ny, fix_dist + int2fix15(1)
                ));
345
346             ball->vx = ball->vx + (multfix15(nx, intermediate));
347             ball->vy = ball->vy + (multfix15(ny, intermediate));
348
349             // Only trigger audio and bounciness scaling once per peg per
            ball per contact
350             if (curr_peg != ball-> last_peg) {
351

```

```

352         playSound_once();
353
354         ball->vx = multfix15(bounciness_g, ball->vx);
355         ball->vy = multfix15(bounciness_g, ball->vy);
356     }
357     ball->last_peg = curr_peg;
358 }
359 }
360 else {
361     ball->last_peg = -1;
362 }
363 }
364
365 // find_max used to scale histogram drawing; returns max value in
366 // histogram
367 int find_max(int *arr) {
368     int result = arr[0];
369     for (int i = 0; i < NUM_BINS; i++) {
370         result = (arr[i] > result) ? arr[i] : result;
371     }
372     return result;
373 }
374
375 // === Serial thread =====
376 static PT_THREAD (protothread_serial(struct pt *pt))
377 {
378     PT_BEGIN(pt) ;
379     PT_YIELD_usec(1000000) ;
380     sprintf(pt_serial_out_buffer, "Galton Board Simulation v2.0 with Audio
381 \n\r") ;
382     serial_write ;
383     while(1) {
384         sprintf(pt_serial_out_buffer, "Commands: 'r'=reset ball, 's'=stats
385 : ") ;
386         serial_write ;
387         serial_read ;
388         if (pt_serial_in_buffer[0] == 'r' || pt_serial_in_buffer[0] == 'R'
389 ) {
390             // spawnBalls(numballs) ;
391             sprintf(pt_serial_out_buffer, "Ball reset!\n\r") ;
392             serial_write ;
393         }
394         else if (pt_serial_in_buffer[0] == 's' || pt_serial_in_buffer[0]
395 == 'S') {
396             // sprintf(pt_serial_out_buffer, "Ball pos: (%d,%d) vel: (%d,%
397 d) gravity: %f\n\r",
398             //         fix2int15(ball_x), fix2int15(ball_y),
399             //         fix2int15(ball_vx), fix2int15(ball_vy), fix2float15(
400 GRAVITY)) ;
401             // serial_write ;
402         }
403     }
404     PT_END(pt) ;
405 }

```

```

399
400 float bounce_float; // store bounciness as float for display
401
402 // === ADC / UI protothread =====
403 // Periodically reads ADC and maps to either number of balls or bounciness
    depending on ui_state
404 static PT_THREAD (protothread_adc_read(struct pt *pt))
405 {
406     PT_BEGIN(pt) ;
407     PT_YIELD_usec(1000000) ;
408     int previous;
409     uint16_t result;
410     while (1) {
411         previous = result;
412         result = adc_read();
413
414         if (ui_state == UI_SET_BALLS){ // Change number of balls only when
            in SET BALLS state
415             int target = map_adc_to_balls(result);
416             if (target != numballs){ // Change number of balls if adjusted
                number != current number
417                 numballs = target;
418             }
419         }else if (ui_state == UI_SET_BOUNCE){
420             // map ADC to bounciness (Q15) and also store as float for
                display
421             bounciness_g = map_adc_to_bounce(result); // Change the
                bounciness
422             bounce_float = fix2float15(bounciness_g); // Convert
                bounciness to float to pass into VGA Display
423         }
424         PT_YIELD_usec(50000);
425     }
426
427
428     PT_END(pt) ;
429 }
430
431 // === Button parameters =====
432 #define BUTTON_PIN 10
433 #define LED_PIN 25
434
435 #define DEBOUNCE_SAMPLES 4
436
437 // protothread that implements software debounce and cycles UI state on
    confirmed presses
438 static PT_THREAD (protothread_button(struct pt *pt))
439 {
440     PT_BEGIN(pt);
441     gpio_init(BUTTON_PIN);
442     gpio_set_dir(BUTTON_PIN, GPIO_IN);
443     gpio_pull_up(BUTTON_PIN);
444
445

```

```

446 // FSM bookkeeping (match your keypad style)
447 static int STATE_1 = 0;           // 0:not pres, 1:maybe, 2:press,
                                     3:maybe rel
448 static int possible = 1;         // previous press we're checking
                                     (1=released)
449 static int stable = 1;
450 static int same_cnt = 0;
451
452 while (1)
453 {
454     int i = gpio_get(BUTTON_PIN);
455
456     switch (STATE_1) {
457         // ----- (not pressed) -----
458         case 0:
459             if (i != 1) {
460                 possible = i;
461                 same_cnt = 1;
462                 STATE_1 = 1;
463             }
464             break;
465         // -----(maybe pressed) -----
466         case 1:
467             if (i == possible) {           // same level
468                 if (++same_cnt >= DEBOUNCE_SAMPLES) {
469                     // KEY PRESSED (confirmed) !!!
470                     stable = possible;     // 0
471
472                     ui_state = (ui_state == UI_IDLE) ? UI_SET_BALLS
473                               : (ui_state == UI_SET_BOUNCE) ? UI_IDLE
474                               : UI_SET_BOUNCE;
475                     printf("UI state -> %d\n", ui_state);
476
477                     STATE_1 = 2;           // Now in pressed state
478                     same_cnt = 0;
479                 }
480             } else {                       // bounce back
481                 STATE_1 = 0;              // Not pressed
482                 same_cnt = 0;
483             }
484             break;
485
486         // ----- (pressed) -----
487         case 2:
488             if (i != 0) {                 // (possible release?)
489                 possible = i;             // should be 1
490                 same_cnt = 1;
491                 STATE_1 = 3;              // Maybe not pressed
492             }
493             break;
494
495         // ----- RISING (maybe released) -----
496         case 3:
497             if (i == possible) {

```

```

498         if (++same_cnt >= DEBOUNCE_SAMPLES) {
499             // KEY RELEASED confirmed
500             stable = possible;        // 1
501             STATE_1 = 0;              //Ready for next press
502             same_cnt = 0;
503         }
504     } else {
505         STATE_1 = 2;
506         same_cnt = 0;
507     }
508     break;
509 }
510
511     PT_YIELD_usec(10000);
512 }
513
514 PT_END(pt);
515 }
516
517 // === Histogram / drawing protothread =====
518 // Maintains histogram counts and draws histogram bars on VGA display
519 static PT_THREAD (protothread_hist(struct pt *pt)) {
520     PT_BEGIN(pt) ;
521
522     for (int b = 0 ; b < NUM_BINS; b++) {
523         histogram[b] = 0;
524     }
525
526     int hist_start_x = 0;
527     int hist_start_y = ARENA_BOTTOM + 5;
528     int bin_width = (640) / NUM_BINS;
529     prev_balls = numballs;
530     prev_bounce = bounciness_g;
531
532     while(1) {
533
534         if (abs(numballs - prev_balls) > (numballs / 20) || abs(
            prev_bounce - bounciness_g) > (divfix(bounciness_g, int2fix15
            (20)))) {
535             printf("RESET STATS - bounce %f, prev_bounce %f\n",
                fix2float15(bounciness_g), fix2float15(prev_bounce));
536             prev_balls = numballs;
537             prev_bounce = bounciness_g;
538             PT_RESTART(pt);
539         }
540
541         int max_histogram = find_max(histogram);
542
543         clearLowFrame(ARENA_BOTTOM, 0);
544
545         for (int i = 0; i < NUM_BINS; i++) {
546             int bar_height = ((float) histogram[i] / ((float)
                max_histogram)) * 90;    // Scale factor

```

```

547         // printf("BAR HEIGHT %i: %i, Hist: %i\n", i, bar_height,
           histogram[i]);
548         int x = hist_start_x + (i * bin_width);
549         drawRect(x, hist_start_y + (90 - bar_height), bin_width - 2,
           bar_height, 1);
550     }
551
552     // for (int x = 0; x < NUM_BINS; x++ ) {
553     //     printf("%i : %i - ", x, histogram[x]);
554     // }
555     // printf("\n");
556
557     prev_balls = numballs;
558     prev_bounce = bounciness_g;
559
560     PT_YIELD_usec(10000);
561 }
562 PT_END(pt) ;
563 }
564
565 // === Drawing helpers =====
566 // Small filled circle approximations and tiny ball drawing utilities
567
568 // ultraDrawBall draws a 2x2 pixel block; used for fast erase/draw updates
569 // It's used during animation updates for rapid erase/redraw operations.
570 void ultraDrawBall(short x0, short y0, bool color) {
571     drawPixel(x0, y0, color);
572     drawPixel(x0+1,y0, color);
573     drawPixel(x0, y0+1, color);
574     drawPixel(x0+1,y0+1, color);
575 }
576
577 // Draws a larger filled circle (~13x13) centered at (x0, y0).
578 // Approximated using multiple vertical lines for smooth edges.
579 // Typically used to draw pegs in the Galton board grid.
580 void fillCircle6(short x0, short y0, char color) {
581     drawVLine(x0, y0 - 6, 13, color);
582     drawVLine(x0 + 1, y0 - 6, 13, color);
583     drawVLine(x0 + 6, y0 - 1, 3, color);
584     drawVLine(x0 - 1, y0 - 6, 13, color);
585     drawVLine(x0 - 6, y0 - 1, 3, color);
586     drawVLine(x0 + 2, y0 - 6, 13, color);
587     drawVLine(x0 + 6, y0 - 2, 5, color);
588     drawVLine(x0 - 2, y0 - 6, 13, color);
589     drawVLine(x0 - 6, y0 - 2, 5, color);
590     drawVLine(x0 + 3, y0 - 5, 11, color);
591     drawVLine(x0 + 5, y0 - 3, 7, color);
592     drawVLine(x0 - 3, y0 - 5, 11, color);
593     drawVLine(x0 - 5, y0 - 3, 7, color);
594     drawVLine(x0 + 4, y0 - 4, 9, color);
595     drawVLine(x0 - 4, y0 - 4, 9, color);
596 }
597
598 // These specific constants are precomputed to avoid runtime conversion

```

```

599 // when performing fixed-point division and coordinate alignment in the
600 // animation loop (for peg grid spacing and row/column indexing).
601 #define FIX15_6p5 0x34000
602 #define FIX15_38 0x130000
603 #define FIX15_19 0x98000
604 #define FIX15_22 0xb0000
605
606 // === Animation thread =====
607 // This protothread updates all ball positions, detects peg and wall
608 // collisions, handles ball respawning, and refreshes the VGA display.
609 // Runs periodically based on FRAME_RATE.
610 static PT_THREAD (protothread_anim(struct pt *pt))
611 {
612     PT_BEGIN(pt) ;
613
614     static int begin_time ;
615     static int spare_time ;
616     static int prev_ball_x[NUM_BALLS], prev_ball_y[NUM_BALLS];
617     static int i,b;
618     static uint32_t prev_second = 0;
619
620     int px;
621     int py;
622     int prox_row;
623     int prox_col;
624
625     // Initialize previous ball positions for the first frame
626     for ( b = 0; b < numballs; b++ ) {
627         prev_ball_x[b] = fix2int15(balls[b].coord.x) ;
628         prev_ball_y[b] = fix2int15(balls[b].coord.y) ;
629     }
630     while(1) {
631
632         begin_time = time_us_32() ;
633
634         // Increment boot-time counter once per second
635         if ((begin_time - prev_second) >= 1000000) {
636             boot_time_seconds++;
637             prev_second = begin_time;
638         }
639
640         // === Update physics for each ball ===
641         for ( int b = 0; b < numballs; b++ ) {
642             balls[b].vy = balls[b].vy + GRAVITY ;
643
644             struct Ball ball = balls[b];
645
646             py = fix2int15(divfix(ball.coord.y + FIX15_6p5, FIX15_19));
647             int c = (py % 2) * 19; // row offset
648             px = fix2int15(divfix(ball.coord.x + FIX15_22 - int2fix15(c),
649                                 FIX15_38));
650
651             // If ball is near a peg (using proximity lookup), handle
652             // collision

```



```

651     if (prox_arr[px][py].val) {
652         handleCollisionResponse(peg_array[prox_arr[px][py].peg_i],
                                &balls[b], i) ;
653     }
654
655     // Integrate motion
656     balls[b].coord.x = balls[b].coord.x + balls[b].vx ;
657     balls[b].coord.y = balls[b].coord.y + balls[b].vy ;
658
659
660     if (hitLeft(balls[b].coord.x)) {
661         // Bounce off left wall with damping
662         balls[b].coord.x = int2fix15(ARENA_LEFT + BALL_RADIUS) ;
663         balls[b].vx = multfix15(-balls[b].vx, bounciness_g) ;
664     }
665     if (hitRight(balls[b].coord.x)) {
666         // Bounce off right wall
667         balls[b].coord.x = int2fix15(ARENA_RIGHT - BALL_RADIUS) ;
668         balls[b].vx = multfix15(-balls[b].vx, bounciness_g) ;
669     }
670     // === Ball reaches bottom ===
671     if (hitBottom(balls[b].coord.y)) {
672         balls_fallen++;
673
674         // Update histogram (distribution of final x-positions)
675         int bin = getBin(balls[b].coord.x);
676         histogram[bin]++;
677         // Respawn ball at top with randomized velocity
678         balls[b].coord.x = int2fix15(SPAWN_X) ;
679         balls[b].coord.y = int2fix15(SPAWN_Y) ;
680         balls[b].vx = (rand() & 0x0000ffff) - int2fix15(1);
681         balls[b].vy = int2fix15(0) ;
682         balls[b].last_peg = -1;
683     }
684 }
685
686 // === VGA Rendering ===
687 // Clear old ball positions beyond active count
688 for ( b = numballs; b < MAX_BALLS; b++) {
689     ultraDrawBall(prev_ball_x[b], prev_ball_y[b], 0);
690 }
691 for ( b = 0; b < numballs; b++ ) {
692     int current_ball_x = fix2int15(balls[b].coord.x) ;
693     int current_ball_y = fix2int15(balls[b].coord.y) ;
694     ultraDrawBall(prev_ball_x[b], prev_ball_y[b], 0) ;
695     ultraDrawBall(current_ball_x, current_ball_y, 1) ;
696     prev_ball_x[b] = current_ball_x ;
697     prev_ball_y[b] = current_ball_y ;
698 }
699 for (i = 0; i < NUM_PEGS; i++) {
700     fillCircle6(fix2int15(peg_array[i].x), fix2int15(peg_array[i].
701         y), 1);
702 }

```

```

703     //printf("Spare time: %i\n", spare_time);
704     spare_time = FRAME_RATE - (time_us_32() - begin_time) ;
705     if (spare_time > 0) {
706         PT_YIELD_usec(spare_time) ;
707     }
708 }
709 PT_END(pt) ;
710 }
711
712
713 // === Statistics Display Thread =====
714 // Updates on-screen system stats such as number of balls, bounciness,
715 // total balls fallen, and elapsed runtime. Refreshes every 10 ms.
716 static PT_THREAD (protothread_stats(struct pt *pt)) {
717     PT_BEGIN(pt);
718
719     static char stats_buffer[32];
720
721     while(1) {
722         // This is the code for printing the numbers
723         fillRect(450, 10, 180, 60, 0); // I added this so the writing
724             becomes clear,
725
726         // Display current number of active balls
727         sprintf(stats_buffer, "Active: %lu", numballs);
728         setCursor(460, 15);
729         setTextColor(GREEN);
730         setTextSize(1);
731         writeString(stats_buffer);
732
733         // Display how many balls have fallen through the board
734         sprintf(stats_buffer, "Balls Fallen: %lu", balls_fallen);
735         setCursor(460, 30);
736         setTextColor(GREEN);
737         setTextSize(1);
738         writeString(stats_buffer);
739
740         sprintf(stats_buffer, "Bounciness: %f", fix2float15(bounciness_g))
741             ;
742         setCursor(460, 45);
743         setTextColor(GREEN);
744         writeString(stats_buffer);
745
746         // Display current bounciness coefficient (converted from Q15)
747         uint32_t hours = boot_time_seconds / 3600;
748         uint32_t minutes = (boot_time_seconds % 3600) / 60;
749         uint32_t secs = boot_time_seconds % 60;
750         sprintf(stats_buffer, "Time: %02lu:%02lu:%02lu", hours, minutes,
751             secs);
752         setCursor(460, 60);
753         setTextColor(GREEN);
754         writeString(stats_buffer);
755
756         PT_YIELD_usec(10000);

```

```

754     }
755
756     PT_END(pt);
757 }
758
759
760 // === Main =====
761 // Initializes system peripherals, spawns threads, and starts scheduler.
762 int main(){
763     set_sys_clock_khz(250000, true) ;
764     stdio_init_all() ;
765     audio_init_dma();      // Initialize audio output
766     initVGA() ;
767     stdio_init_all();
768     // printf("ADC Example, measuring GPIO26\n");
769     adc_init();
770     init_prox_arr();
771
772     // Make sure GPIO is high-impedance, no pullups etc
773     adc_gpio_init(26);
774     // Select ADC input 0 (GPIO26)
775
776     rand_state = time_us_32() ;
777     srand(rand_state);
778     // Spawn initial set of balls
779     spawnBalls(NUM_BALLS) ;
780
781     adc_select_input(0);
782     // Generate peg layout
783     generate_pegs(16, 320, 60);
784
785     // fillRect(0, 0, 640, 480, 1);
786
787     // int r = 0;
788     // while(1) {
789     //     memset(vga_data_array, 0xff, TXCOUNT);
790     // }
791
792     // Register all protothreads
793     pt_add_thread(protothread_serial);
794     pt_add_thread(protothread_button);
795     pt_add_thread(protothread_anim);
796     pt_add_thread(protothread_adc_read);
797     pt_add_thread(protothread_hist);
798     pt_add_thread(protothread_stats);
799     // Start Scheduler
800     pt_schedule_start ;
801 }

```